
PROJECT REFERENCE GUIDE

MCQ Shuffler Pro Card & Code Map

Every card, every button, every mother & child in the codebase — with exact file paths, line numbers and step-by-step edit recipes. Say "edit card X" and change only that piece.

23 UI components

8 shared mothers

6 edit playbooks

One-place theme control

3

WORK MODES

2

DOWNLOAD FORMATS

530+

UNIT TESTS

10

E2E SUITES

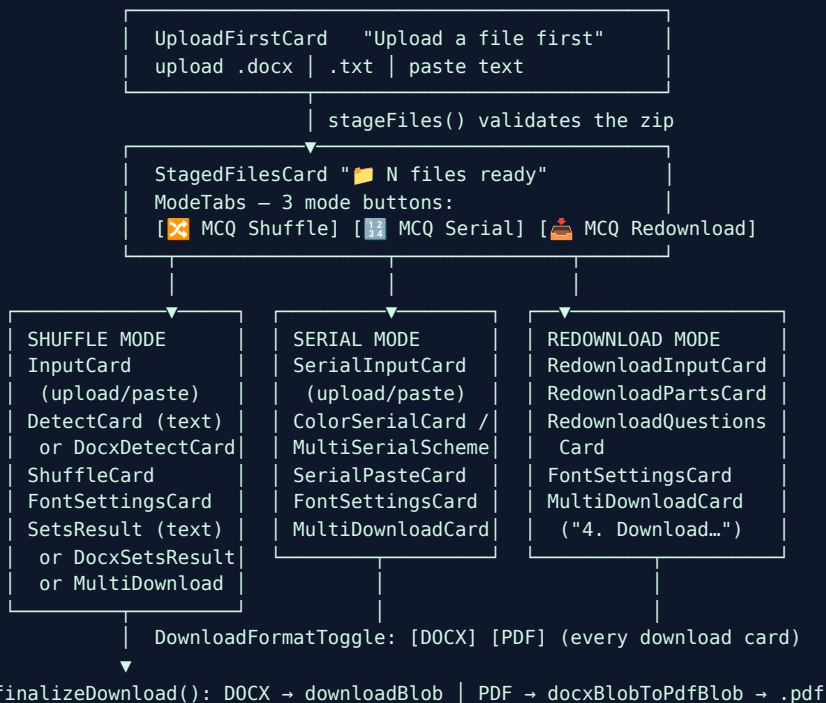
This guide is an **edit-locator**: it maps every visible card and button on **mcq-shuffler-pro** (https://afnan-samin.github.io/MCQ_Shuffler_Pro/) to the exact file and line where it lives. The project was deliberately re-shaped around a **mother / child** architecture, so a small, targeted edit almost never touches the whole project.

Golden rule — find the card in **Part 3**, open its file, change only that card's markup or text. Buttons push events up to `src/app/page.tsx`, which is the single wiring layer: UI lives in `src/components/mcq/`, all logic lives in `src/lib/mcq/`. If your edit changes a visible label, sync the E2E selectors listed in the Appendix in the same commit.

Conventions used: **M** = **mother** (shared component used by several modes — edit it and every consumer changes); **c** = **child** (belongs to one mode only). Line numbers were verified against the current codebase (commit series ending `0d8188e`). All button labels are quoted byte-exact from code so you can `rg`-search for them.

PART 1 Site flow at a glance

The user journey has four beats: **upload** → **mode selection** → **work view** → **download**. After staging files, exactly three mode buttons appear; each mode owns its own input, option and result cards; and every download card carries the same two shared widgets — the font settings card and the DOCX/PDF format toggle.



While inside any mode, a persistent top bar (**ModeWorkBar**) shows ◀ **back home** · mode title · **N/M files** · **Add files**. Under every download card sits **NextModesCard** ("✅ Keep working with

these files") which carries the loaded files into the other modes without re-uploading.

PART 2 The Mothers — shared infrastructure

Mothers are the pieces every mode leans on. Editing a mother changes **every consumer at once** — that is their purpose. Treat `repack-docx.ts`, `docx-xml.ts` and `parser.ts` as protected mothers: they guarantee the "formatting kept exactly" promise, and any edit there must run the full unit-test suite (`scripts/test-*.ts`, 530+ assertions).

FileDropzone M `src/components/mcq/file-dropzone.tsx` 207 lines

Exports: `FileDropzone` ({`accept`, `multiple`, `disabled`, `busy`, `promptText`, `hintText`, `icon`, `variant`: `"lg"` | `"md"` | `"compact"`, `inputTestId`, `maxSizeBytes`, `maxFiles`, `onFiles`, `triggerRef`, `chips`, `children`}) (line 75) · `DropzoneTrigger` {`open`} (19, via `useImperativeHandle`) · `DropzoneRejection` {`notAccepted`, `tooBig`, `overMax`} (23).

What: the ONE drag-drop + click-to-browse upload surface. Filters extension → size → count and reports rejects through `onFiles` (`files`, `rejected`). **Consumers (5 components / 6 instances):** `upload-first-card` (110), `input-card` (116), `serial-input-card` (91 + 146 "Add more files"), `redownload-input-card` (48, `compact`), `mode-work-bar` (66, `children-slot`). **Edit impact:** change upload prompts/limits look here once — every surface updates.

runFilePipeline M `src/lib/mcq/file-pipeline.ts` 150 lines

Exports: `runFilePipeline` (`files`, {`parse`, `maxBytes?`}) → {`items`, `failures`, `tooBig`, `notDocx`} (73) — the single loader used by all 3 modes (50 MB guard → extension guard → `loadDocxXml` → per-mode parse callback). `isValidDocxZip` (`file`) (33) — light JSZip check that `word/document.xml` exists; runs at staging time so fake docx files never enter a mode. `FILE_TOO_BIG_MSG` (24) · `docxBaseName` (43) · `prepareShuffleXml` (`originalXml`) (125) — strips color headers + non-MCQ lines for shuffle.

Consumers: `page.tsx` `stageFiles` (305), `handleDocxFile` (629), `handleShuffleFiles` (1360), `loadSerialFiles` (717), `loadRedownloadFiles` (1155). **Edit impact:** validation/loading behavior changes hit every mode at once; toast texts live in `page.tsx`, not here.

MODE_META M `src/lib/mcq/mode-meta.ts` 56 lines

Exports: `McqMode` = `"shuffle"` | `"serial"` | `"redownload"` (7) · `MODE_META` (25) — the single registry: emoji (🗂️/📄/📂), `title`, `tabTitle`, `description`, `ariaLabel`, `icon` · `MODE_IDS` (56) = render order.

Consumers: `mode-tabs` (4), `next-modes-card` (5), `mode-work-bar` (7), `page.tsx`. **Edit impact:** rename a mode or change its emoji here → tabs, work-bar title and next-mode buttons all update together. **Do NOT rename the id strings** ("shuffle"...) — they are stored in `localStorage` (`mcq-shuffler-mode`) and used as switch keys.

repackDocx / repackDocxRemapped

M[src/lib/mcq/repack-docx.ts](#)

81 lines

Exports: `repackDocx(source, parts, extra?, mimeType?)` (24) — copies every non-directory zip entry byte-identical, replaces only `parts`. `repackDocxRemapped(source, documentXml, fontSettings?, extraParts?, extra?, mimeType?)` (56) — same + optional font remap of `word/document.xml` and `word/styles.xml`; disabled ⇒ byte-identical. `STYLES_XML_PATH` (16), `DOCX_MIME` (13).

Consumers: `multi-docx.ts`, `docx-exporter.ts`, `color-serial.ts`, `exporter.ts`. **Edit impact:** THE core of "formatting kept exactly" — every docx output in every mode passes through it. Never edit casually; run all tests.

applyFontRemap — font engine

M[src/lib/mcq/font-remap.ts](#)

320 lines

Exports: `applyFontRemap({documentXml, stylesXml?}, settings, options?)` (308) · `applyFontRemapXml` (245) — per-`<w:r>` font remap, idempotent, text untouched · `applyFontRemapStylesXml` (286) — legacy Bijoy font values (Sutonny*/Bijoy*/Shibly*) → chosen Bijoy font · `classifyRunText` (108) → "bijoy" | "unicode-bengali" | "latin" · `FontSettings` (53) · `FONT_CHOICES` (63) — the 5+5+5 dropdown options · `DEFAULT_FONT_REMAP_SETTINGS` (55) = Times New Roman / SutonnyMJ / Noto Serif Bengali, enabled.

Consumers: `repack-docx.ts` (the wiring), `exporter.ts`, `font-settings-card.tsx`, `page.tsx` (`sanitizeFontSettings` 107 validates against `FONT_CHOICES`). **Edit impact:** add a font by appending to `FONT_CHOICES` — the dropdown and the sanitizer pick it up automatically.

siteTheme — one-place theme control

M[src/config/theme.ts](#)292 lines · + `theme-css.ts` 106 lines

What: the single file that controls **all site fonts, colors, popups and corner radius**. `siteTheme` (142): `fonts` (sans/serif/mono/bengali), `colors.light` + `colors.dark` (background, card, primary, muted, destructive + destructiveForeground, border, ring, brand 50–950), `radius.base` (one knob, sm/md/lg/xl derived), `popup` (radius/shadow/border/backdropBlur). `theme-css.ts` turns it into `:root {--st-*}` + `.dark {--st-*}` CSS; `src/app/layout.tsx` (67) injects the tag; `globals.css` re-points all Tailwind/shadcn tokens at those vars.

Iron rule: edit `theme.ts` ONLY — never edit `theme-css.ts` or the var chain. The file's bottom comment carries the three official recipes:

① **Site font everywhere** — edit `fonts.sans` (147), e.g. `'Verdana, "Noto Sans Bengali", sans-serif'`; for a web font add it via `next/font` in `layout.tsx` with `variable: "--font-myfont"`, put it on `<html>`, then use `sans: 'var(--font-myfont), ...'`. Keep a Bengali-capable font in the stack. ② **Brand color** — edit `colors.light.brand` 600 (solid fills) + 700 (hover), tune neighbors; every `bg-brand-600` / `text-brand-700` class site-wide follows. ③ **Popups** — edit `popup.radius` ("0px" sharp / "12px" round) and `popup.shadow`; toasts + select dropdowns follow.

Exports: `DownloadFormat = "docx" | "pdf"` (35) · `PDF_MAX_PAGES = 300` (40, memory guard with English abort message) · `pdfFileNameOf(name)` (50) · `docxBlobToPdfBlob(docx, baseName)` (80) — 100% in-browser: docx-preview render → html2canvas-pro capture (scale $2 \leq 100$ pages, $1.5 \leq 300$) → jsPDF A4; canvases taller than A4 are sliced one-page-per-A4-chunk so continuous documents paginate correctly.

Consumers: `page.tsx` `finalizeDownload` (376) + `finalizeZipEntries` (387) only. **Edit impact:** all PDF quality/limit behavior lives here; the DOCX path never touches this file.

The other lib files — mothers vs mode helpers at one glance

FILE (LINES)	ROLE — MOTHER = SHARED, CHILD = SINGLE-MODE HELPER
<code>exporter.ts</code> (372)	Mother (text pipelines). Fresh .docx via the <code>docx</code> lib for text-flow sets & serial-paste: <code>buildSetsDocxBlob</code> (202), <code>maybeRemapPackedDocx</code> (192), <code>exportDocHtml</code> (299, .doc), <code>printSets</code> (306), <code>downloadBlob</code> (356), <code>DEFAULT_EXPORT_OPTIONS</code> (37).
<code>multi-docx.ts</code> (587)	Mother (multi-file). <code>replaceDocumentXml</code> (268) swaps document.xml inside the original zip; <code>buildMergedDocxBlob</code> (414) merges files with page breaks (one container-level font remap); <code>buildZipBlob</code> (556); <code>offsetSerialPlan</code> (583).
<code>docx-xml.ts</code> (696)	Mother (docx engine). <code>loadDocxXml</code> (537), <code>parseDocxXml</code> (426 — questions/serial/options/answers/explanations), Bengali/Bijoy digit maps, <code>renumberSerialPara</code> (645). Contains functional Bengali keywords — edit carefully, tests guard it.
<code>parser.ts</code> (243)	Mother (text engine). <code>parseMcq</code> (141), <code>analyzeSerial</code> (196), <code>autoFixNumbering</code> (223) for paste/.txt flows. Question-start regex is functional Bengali — change only with unit tests.
<code>set-engine.ts</code> (132)	Mother (shuffle sets). <code>buildSets</code> (46; interleaved / chunk / random / original), <code>getSetName</code> (93 — writes সেট A / সেট ক / সেট ১ / Set 1 into output files), <code>shuffled</code> (17).
<code>docx-exporter.ts</code> (206)	Child (shuffle, also serial-fix). <code>buildShuffledXml</code> (90 — DOM-level XML surgery: sets, page breaks, headers, renumber, reference tags), <code>buildShuffledDocxBlob</code> (170), <code>buildSerialFixedDocxBlob</code> (190).
<code>color-serial.ts</code> (733)	Child (serial mode; used by shuffle prep). <code>analyzeColorDocx</code> (231), <code>planSerialByColor</code> (472), <code>applyColorSerialXml</code> (648), <code>buildColorSerialDocxBlob</code> (720), <code>stripShadedParasXml</code> (315), <code>stripNonMcqLinesXml</code> (383).
<code>redownload.ts</code> (793)	Child (redownload). <code>parseRedownloadXml</code> (259 — six-part detection), <code>PART_LABELS</code> (49), <code>buildRedownloadXml</code> (511), <code>extractWatermark</code> (761).
<code>reference.ts</code> (455)	Child (shuffle). Detects <code>[CU-A: 22-23]</code> source tags; <code>analyzeRefReport</code> (304), <code>buildRefEditedMap</code> (442) for keep / strip / end-line modes.
<code>encoding.ts</code> (211)	Mother-ish (rendering). Word-level Bijoy/Unicode/English classifier behind TokText preview: <code>classifyWord</code> (61), <code>tokenizeWithContext</code> (100).
<code>serial-paste.ts</code> (28)	Child (serial paste). <code>renumberQuestionsByPosition</code> (17).
<code>limits.ts</code> (30)	Mother (limits). <code>MAX_SERIAL_NUMBER = 9999</code> (18) · <code>MAX_FILE_BYTES = 50 MB</code> (24) · <code>SHUFFLE_MAX_FILES = 50</code> (30).

Every UI card in the project, alphabetically by file. **"UI name"** is the exact English title the user sees; the tables quote every button label byte-exactly from code, and the **Code path** column traces handler → logic so an edit can be traced end-to-end in seconds.

ColorSerialCard c `src/components/mcq/color-serial-card.tsx` 160 ln

UI name: "Color-based serial (structured file)" (62) · **Where:** serial mode, exactly 1 file with colored headers (page 1796) · **Testids:** none. Color chips 83–126, action button 153–156.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
color chip "<name> · N sections"	selects that color's serial scheme	local sel state (setSel)
Continuous across the file	selects the continuous scheme	local setSel({kind:"continuous"})
Download serial .docx (B1) – or .pdf	plans serial by color, applies it, downloads	onSerial → handleColorSerial (page 786) → planSerialByColor (472) → buildColorSerialDocxBlob (720) → finalizeDownload (376)
DownloadFormatToggle	DOCX/PDF choice	onFormatChange → updateDownloadFormat (page 363)

DetectCard c `src/components/mcq/detect-card.tsx` 327 ln

UI name: "Detection result & question selection" (98) · **Where:** shuffle work view, TEXT flow only — pasted text or .txt (page 2004) · **Testids:** `id="step-detect"` (94), switch `id="allow-broken"` (235). Serial status 185–243, toolbar 246–275, list 278–323.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Select all / Deselect all	selects / clears every parsed question	onSelectAll / onSelectNone → selectAll (1099) / selectNone (1104)
Select range (From / To inputs)	selects a position range; accepts Bengali or English digits	onSelectRange → selectRange (1106) → digitsToNumber (docx-xml 35)
 Fix numbering automatically (busy: Fixing...)	renumbers pasted text 1..N and re-parses	onAutoFix → handleAutoFix (580) → autoFixNumbering (parser 223)
Run as-is – Switch	allows shuffling with a broken serial	onAllowBrokenChange → setAllowBroken (page state)
checkbox per question	ticks / unticks that question	onToggle → toggleQuestion (599)
Show more (N remaining)	paginates the list by 100	local setVisible

DocxDetectCard (+ SerialSpan)

C

src/components/mcq/docx-detect-card.tsx

385 ln

UI name: "Detection result — {fileName}" (106) · **Where:** shuffle work view, single-docx flow (page 1945) ·

Testids: `id="step-detect"` (102), switch `id="docx-allow-broken"` (281). SerialSpan (38) is reused by docx-sets-result. Encoding detector 142–197, serial status 213–290, toolbar 293–322.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Select all / Deselect all / Select range	selection helpers (Bengali + English digits in range)	page selectAll (1099) / selectNone (1104) / selectRange (1106)
 Fix serial & download .docx (1..N) (busy: Building...)	downloads a serial-fixed copy — original order, 1..N	onSerialFix → handleDocxSerialFix (1042) → buildSerialFixedDocxBlob (docx-exporter 190) → repackDocxRemapped → finalizeDownload
Run as-is – Switch	needed when numbering restarts per section	onAllowBrokenChange → setAllowBroken
checkbox per question · Show more (N remaining)	tick / untick · paginate by 50	toggleQuestion · local setVisible

DocxSetsResult

C

src/components/mcq/docx-sets-result.tsx 172 ln

UI name: "Shuffle complete — N set(s) (M questions)" (66) · **Where:** shuffle work view after a single-docx

shuffle (page 1984) · **Testids:** `id="step-result"` (61), switch `id="renumber-switch"` (86). Export bar 95–104, set cards 107–155.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Serial replace ON – 1, 2, 3... / OFF – original numbers (Switch)	toggles renumbering for downloads AND set badges	onReNUMBERChange → toggleSerialByClick (page 712)
 Word (.docx) – renumbered serials (1,2,3...) /  PDF variant	builds + downloads the renumbered version	onDownload(true) → handleDocxDownload (1013) → buildShuffledDocxBlob (170) → finalizeDownload
 Word (.docx) – original numbers /  PDF variant	original serials version	onDownload(false) → same handler
Copy / Copied (per set) · clicking a serial number	copies that set's text · toggles serial replace	onCopySet → handleDocxCopySet (1067) · onReNUMBERChange(!reNUMBER)
DownloadFormatToggle	DOCX/PDF choice	updateDownloadFormat (page 363)

DownloadFormatToggle

M[src/components/mcq/download-format-toggle.tsx](#)

71 ln

UI name: "Download as" + pills **DOCX** / **PDF** (27/46/61) · **Where:** inside all 6 download cards — DocxSetsResult (91), SetsResult (224), SerialPasteCard (139), ColorSerialCard (151), NoColorSerialCard (serial-extra-cards 96), MultiDownloadCard (104). State lifted to page.tsx, persisted in localStorage (`mcq-download-format`), DOCX default.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
DOCX pill	default; buttons show "Word (.docx)" wording	<code>onChange("docx")</code> → <code>updateDownloadFormat</code> (363)
PDF pill	download buttons swap to ".pdf / PDF" labels; hint "PDF is rendered in your browser (best effort)" appears (66)	<code>onChange("pdf")</code> → page 363; conversion in <code>pdf-export.ts</code> (80)

Testids: `data-testid="download-format"` (24), `format-docx` (35), `format-pdf` (50); `aria-pressed` on both pills.

FontSettingsCard

M[src/components/mcq/font-settings-card.tsx](#)

198 ln

UI name: "Fonts in the output file" (103) — collapsed by default; description reads "Remap ON — Unicode → X, Bijoy → Y, English → Z" or "Remap is OFF — the output keeps the original fonts" (105–107) · **Where:** rendered exactly once per view just above the download area: redownload (1734), serial (1791), shuffle-multi (1910), shuffle single-docx (1980), text flow (2037) · **Testids:** `data-testid="font-settings-card"` (93), `id="font-settings-content"` (127).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
header click / chevron	expand / collapse; syncs the draft to applied settings	<code>local toggleOpen</code> (69)
Remap fonts in the output file – Switch	draft remap ON/OFF	<code>patch({enabled})</code> → committed by <code>onChange</code>
English fonts / Bijoy-ANSI fonts / Unicode-Bengali fonts (selects)	choose from the 5+5+5 FONT_CHOICES lists	<code>patch({englishFont bijoyFont unicodeFont})</code> ; choices from <code>font-remap.ts</code> 63
Use fonts	commits + persists localStorage <code>mcq-font-settings</code> ; flash "Applied — downloads will use these fonts" (190)	<code>confirm()</code> → <code>onChange(draft)</code> → <code>updateFontSettings</code> (page 354)
Reset	restores the default fonts	<code>onChange(DEFAULT_FONT_REMAP_SETTING 5)</code> (<code>font-remap</code> 55)

InputCard

C[src/components/mcq/input-card.tsx](#)

179 ln

UI name: "Add questions — upload a file or paste" (98) · **Where:** shuffle work view top (1849) — serial and redownload have their own input cards · **Testids:** `id="step-input"` (94).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Tab Upload file (108) / Tab Paste (111)	switches input mode	local shadcn Tabs
Click to select a file or drag & drop (122)	docx → shuffle pipeline; .txt → browser parse	onDocxFile (single) / onDocxFiles = handleShuffleFiles (multi) / onTextFileLoaded = loadAndDetect – page 629 / 1360 / 549
 Detect questions (paste mode) (busy: Detecting...)	parses pasted text	onDetect → handleDetect (568) → parseMcq (parser 141)

ModeTabs

M[src/components/mcq/mode-tabs.tsx](#)

64 ln

UI name: the 3 mode buttons (no card title) — labels come from `MODE_META[id].tabTitle`:  MCQ Shuffle ·  MCQ Serial ·  MCQ Redownload (mode-meta 25–53) · **Where:** step 2 only (`flowStep === "select"` with staged input, page 1679); hidden once inside a mode · **Anchors:** `role="tab"` with aria-labels "Shuffle mode" etc.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
each mode button	persists mode, carries staged files into it, enters the work view	onChange(id) → changeMode (397) → carryToMode (469) → flowStep:"work"

ModeWorkBar

M[src/components/mcq/mode-work-bar.tsx](#)

93 ln

UI name: thin work bar — center `"{emoji} {title}"` (48) + hint "— press the arrow to go home" (49), right chip "N/M files" (61) · **Where:** top of every mode's work view (1684); the file cap shows only in shuffle (SHUFFLE_MAX_FILES = 50) · **Testids:** `data-testid="mode-work-bar"` (33), hidden input `data-testid="mode-work-bar-input"` (68).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
◀ (aria-label "Back – return home")	wipes all mode state, returns to the upload card	onBack → backToHome (408)
Add files (88)	opens its own hidden FileDropzone; appends files into the CURRENT mode	onFiles → handleAddMoreFiles (427) → loadSerialFiles / loadRedownloadFiles / handleShuffleFiles

UI name: dynamic `title` prop — "4. Download — new file from picked parts" / "4. Download — picked parts of all files" (redownload, page 1737), "Serial all files together" (serial multi, 1824), "Shuffle complete — download now" (shuffle multi, 1915) · **Where:** results area of all three multi/single-file download views ·

Testids: radios `id="mf-serial-per-file"` / `id="mf-serial-global"` (81/90).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
How should serials work in the merged file? – Restart from 1 in each file / One continuous serial – start to finish	per-file vs global serial (serial multi only)	<code>onSerialStrategyChange</code> → <code>setSerialStrategy</code> (page 827)
Download as one file (.docx) / Download (.docx) when single / .pdf variants	merged single docx/pdf	<code>onDownloadMerged</code> → <code>handleMultiMergedDownload</code> (1496) / <code>handleSerialMultiMerged</code> (827) / <code>handleRdMerged</code> (1265)
Download separately (.zip) – hidden when 1 file	ZIP of per-file outputs	<code>onDownloadZip</code> → <code>handleMultiZipDownload</code> (1527) / <code>handleSerialMultiZip</code> (859) / <code>handleRdZip</code> (1291)
<code>DownloadFormatToggle</code>	DOCX/PDF choice	<code>updateDownloadFormat</code> (363)

ZIP names are decided in `page.tsx`: `MCQ-shuffled-files.zip`, `MCQ-serial-files.zip`, `MCQ-Redownload.zip`.

UI name: file order list (no card; rows with badges "N questions" / "Ready") · **Where:** inside `SerialInputCard` (143), `RedownloadInputCard` (63), and the shuffle-multi files Card rendered inline by `page.tsx` (1874) ·

Anchors: aria-labels are the E2E hooks.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
aria-label "Move up" / "Move down"	reorders files (mobile-friendly arrows)	<code>onReorder(i, i≠1)</code> → <code>reorderSerialDocs</code> (767) / <code>reorderRdDocs</code> (1202) / <code>reorderShuffleItems</code> (1452)
aria-label "Remove from list"	removes that file	<code>onRemove(id)</code> → <code>removeSerialDoc</code> (776) / <code>removeRdDoc</code> (1211) / <code>removeShuffleItem</code> (1464)
whole row draggable	drag-drop reorder	<code>onReorder(from, to)</code> – same handlers

NextModesCard

Msrc/components/mcq/next-modes-card.tsx

58 ln

UI name: "✔ Keep working with these files" (27); button labels = the other modes' tabTitle (same as ModeTabs) · **Where:** under every mode's download card (1753, 1838/1844, 1929, 2000, 2063); renders null when filesCount is 0 (text flow).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
other-mode buttons	carries the current mode's files into the other mode — no re-upload	onOpen(m) → changeMode (397) → carryToMode (469)

RedownloadInputCard

csrc/components/mcq/redownload-input-card.tsx

67 ln

UI name: "1. Upload files — MCQ Redownload" (40) · **Where:** redownload work view (1694) · **Testids:** none.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Click to select a file or drag & drop (compact dropzone)	loads + parses docx files; toasts for too-big and non-docx	onFiles → loadRedownloadFiles (1155) → runFilePipeline + parseRedownloadXml (259) + extractWatermark (761)

RedownloadPartsCard

csrc/components/mcq/redownload-parts-card.tsx

118 ln

UI name: "2. Pick parts — what goes into the new file" (53) · **Where:** redownload work view after files load (1709) · **Anchors:** checkboxes carry `aria-label={PART_LABELS[k]}` (74).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
6 part checkboxes — Serial / Question / Reference / Options / Answer / Explanation (labels from PART_LABELS, redownload.ts 49)	picks what goes into the new file	onChange(kind, v) → setRdParts (1712) → buildRedownloadXml (511) at download time
Renumber serials 1, 2, 3... — Switch (96)	1..N vs original numbers	onRenumberChange → setRdRenumber
hint "Answer expansion on:" (110)	explains answer expansion — auto-shows when Options is off but Answer is on	display only

RedownloadQuestionsCard c

`src/components/mcq/redownload-questions-card.tsx` 219 ln

UI name: "3. Pick questions — {fileName}" (77) — one card per loaded file · **Where:** redownload work view (1719); shows a diagonal watermark overlay when the source docx has one (119–134) · **Testids:** `id="step-rd-questions"` (75).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Select all / Deselect all / Select range	per-file selection helpers (Bengali/English digit range)	<code>selectAllRd</code> (1231) / <code>selectNoneRd</code> (1237) / <code>selectRangeRd</code> (1241)
checkbox per question · Show more (N remaining)	tick / untick · paginate by 50	<code>toggleRdQuestion</code> (1222) · <code>local setVisible</code>

serial-extra-cards — 4 cards c

`src/components/mcq/serial-extra-cards.tsx`

331 ln

1. ColorShuffleInfoCard (14–60) — "This file has colored headers — they'll be stripped for shuffling" (34).

Where: shuffle single-docx flow when the file has colors (1934). Button **Prefer color-based serials? Open in Serial mode** (55) → `openInSerialMode` (961) — moves the file into serial mode.

2. NoColorSerialCard (62–112) — "No colored headers found in this file" (82). Where: serial mode, exactly 1 file without colors (1805). Controls: `DownloadFormatToggle` (96); button **Download continuous 1..N serial .docx/.pdf** (100) → `handleColorSerial({kind:"continuous"}, "continuous")` (786). Shows help text only when `questionCount` is 0.

3. BlockedLinesCard (123–201) — "Stripped lines — N (not going into the shuffle)" (143). Where: shuffle mode when any file had headers/non-MCQ lines stripped (1889, 1942). Controls: **Show N more lines / Show less** (193/189) — local expand.

4. MultiSerialSchemeCard (211–331) — "Per-file serial scheme" (247). Where: serial mode with ≥ 2 files (1818). Controls: per-file **Continuous** chip (285) + one chip per color → `onSchemeChange(id, scheme)` → `setSerialSchemes` (1821).

SerialInputCard

C[src/components/mcq/serial-input-card.tsx](#)

176 ln

UI name: "MCQ Serial — upload files" (71) · **Where:** serial work view top (1759) · **Testids:** hidden main input `data-testid="serial-file-input"` (100).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Tab Upload file (83) / Tab Paste (86)	switches input mode	local shadcn Tabs
Click to select .docx files or drag & drop (97)	loads + color-analyzes docx files	onFiles → loadSerialFiles(fs, false) (717) → analyzeColorDocx (color-serial 231)
 Detect questions (128)	parses pasted text; clears the file list (mutually exclusive)	onPasteDetect → handleSerialPasteDetect (886) → parseMcq
Add more files (161) · MultiFileList (143)	appends to the list · reorder = output order	loadSerialFiles(fs, true) · onReorder/onRemove → 767 / 776

SerialPasteCard

C[src/components/mcq/serial-paste-card.tsx](#)

163 ln

UI name: "Detection result (paste)" (51) · **Where:** serial work view only when files are absent and pasted text was detected (1778) · **Testids:** `id="serial-paste-result"` (47).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
 Fix numbering automatically (busy: Fixing...)	renumbers pasted text 1..N	onFix → handleSerialPasteFix (917) → autoFixNumbering (parser 223)
Download serial .docx (1..N) / .pdf variant	renumbers by position and exports one set	onDownload → handleSerialPasteDownload (935) → renumberQuestionsByPosition (serial-paste 17) → buildSetsDocxBlob (exporter 202) → finalizeDownload
DownloadFormatToggle (139)	DOCX/PDF choice	updateDownloadFormat (363)

UI name: "Shuffle complete — N set(s) (M questions)" (87) · **Where:** shuffle work view after a text-flow shuffle (2041) · **Testids:** `id="step-result"` (82), switch `id="include-header"` (199), datalists `legacy-fonts / unicode-fonts / english-fonts` (132/148/163).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
⚙ Font & header settings (Bijoy/Unicode) / Hide font/header settings (99)	opens the export settings panel	<code>local setShowSettings; options state = page export0pts</code>
Font mode select – Auto – font per word / Bijoy / legacy font / Unicode Bengali / English	export font mode	<code>patch({fontMode}) → buildSetsDocxBlob (exporter 202)</code>
Bijoy font / Unicode Bengali font / English font inputs · Font size (pt)	fonts + size written into the export	<code>patch({legacyFont unicodeFont englishFont fontSize}) → exporter.ts</code>
Set name style – Bengali label সেট A... / Bengali letters সেট ক... / Bengali digits সেট ১... / English – Set 1...	set naming (samples are real output strings written into files)	<code>patch({nameStyle}) → getSetName (set-engine 93)</code>
Put a header on each set's page – Switch + Header text... textarea	per-set page header	<code>patch({includeHeader, headerText}) → exporter.ts</code>
📄 Word (.docx) – one set per page / 📄 PDF variant	main export	<code>onExportDocx → handleExportDocx (1574) → buildSetsDocxBlob → maybeRemapPackedDocx (192) → finalizeDownload</code>
.doc (legacy Word) (235) · 🖨 Print (238) · 📋 Copy all sets (242)	HTML-based .doc · print window · clipboard text (PDF toggle does NOT affect the .doc button)	<code>handleExportDoc (1590) → exportDocHtml (299) · handlePrint (1602) → printSets (306) · handleCopyAll (1623) → allSetsClipboardText (332)</code>
per set: Sort serial / Shuffle (273) · Copy / Copied (283)	re-sorts or reshuffles that set · copies it	<code>onToggleSort → toggleSort (1557) · onCopySet → handleCopySet (1611)</code>
DownloadFormatToggle (224)	DOCX/PDF choice	<code>updateDownloadFormat (363)</code>

UI name: "Shuffle settings & start" (60) · **Where:** shuffle work view in ALL 3 sub-flows — text, single-docx, multi-file (1892, 1962, 2019) · **Testids:** `id="step-shuffle"` (56), input `id="set-count"` (75), radios `dist-original` / `dist-inter` / `dist-chunk` / `dist-random` (122–145), switch `id="shuffle-within"` (164), radios `ref-keep` / `ref-strip` / `ref-endline` (190–204) inside `data-testid="ref-mode-group"` (187).

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
How many sets? – number input + quick chips 3 4 5 10	set count 1–10	<code>onSetCountChange</code> → <code>setSetCount</code> (page state)
Split style – ★ Original Shuffle – all questions in every set / Round-robin (interleaved) / Sequential blocks / Fully random split	distribution mode	<code>onDistributionChange</code> → <code>setDistribution</code> → <code>buildSets</code> (set-engine 46)
Shuffle questions within each set – Switch (hidden for Original)	within-set order	<code>onShuffleWithinChange</code> → <code>setShuffleWithin</code> → <code>buildSets</code>
Handle reference tags – Keep as-is / Strip references – clean question paper / Move to a separate line after each question	reference-tag policy (shown only when tags are detected)	<code>onRefModeChange</code> → <code>setRefMode</code> → <code>buildRefEditedMap</code> (reference 442) at export time
🔀 Shuffle & build sets (busy: Shuffling..., locked icon when disabled)	builds the sets	<code>onShuffle</code> → <code>handleShuffle</code> (1347) → <code>handleTextShuffle</code> (1318) / <code>handleDocxShuffle</code> (985) / <code>handleMultiShuffle</code> (1470)

UI name: none — text render primitive. `TokText({line, dominant, colored=true})` (21) renders a line word-by-word: Bijoy words get the embedded SutonnyMJ web font (`tokfont-bijoy`), `colored` adds per-script classes (`tok tok-bijoy|unicode|english`). **Consumers:** both detect cards, sets-result, docx-sets-result, redownload-questions-card. Colors come from `globals.css` `--st-brand-*` tokens.

UploadFirstCard (+ StagedFilesCard) M

`src/components/mcq/upload-first-card.tsx` 218 ln

UI name: "Upload a file first" (91) — the site's step-1 entry point · **Where:** the only visible card when there is no input at all (page 1665) · **Testids:** `id="step-upload"` (87). Tabs 99–143, detect button 146–154.

BUTTON / CONTROL (EXACT)	WHAT IT DOES	CODE PATH
Tab Upload file (102) / Tab Paste text (105)	switches input mode	<code>local</code> <code>shadcn</code> <code>Tabs</code>
Click to select a file or drag & drop (116)	staging-time validation: every .docx must pass <code>isValidDocxZip</code> else toast "<name> is not a valid .docx"; .txt/.csv capped at 5MB — inline error "Text file too large (5MB max)" (59)	<code>onFiles</code> → <code>stageFiles</code> (305) → <code>isValidDocxZip</code> (file-pipeline 33); .txt → <code>loadAndDetect</code> (549); cap <code>TEXT_MAX_BYTES</code> (24)
 Detect questions (paste mode) (153, busy: Detecting...)	parses pasted text → jumps straight into the shuffle work view	<code>onDetect</code> → <code>handleDetect</code> (568) → <code>parseMcq</code> (parser 141)

StagedFilesCard (166–218, co-exported — there is NO staged-files-card.tsx file): `{N}` files ready — pick a mode now" (185); shown at step 2 above **ModeTabs** (1677). Control: icon-only `aria-label="Clear files"` (195) → `setStagedFiles(null)`.

page.tsx — THE WIRING MOTHER M `src/app/page.tsx` 2080 ln

Role: the single-page app shell and the ONLY stateful layer — every card is presentational and receives handlers as props. Renders the header (1639–1659: hero "MCQ Shuffler Pro" + badges "100% free" / "Data stays in your browser"), the main flow switch (1663–2069), and the footer (2073–2077).

Key state groups: flow — `mode` (175), `stagedFiles` (179), `flowStep` (183) · text input — `rawText`, `parsed` (186–189) · single-docx — `docx` (192) · selection — `selected`, `allowBroken` (196–197) · shuffle config — `setCount`, `distribution`, `shuffleWithin`, `refMode` (201–206) · **fontSettings** (209, persisted `mcq-font-settings`) · **downloadFormat** (212, persisted `mcq-download-format`, DOCX default) · text results — `sets`, `renumber`, `exportOpts` (215–221) · serial — `serialDocs`, `serialSchemes`, `serialPaste*` (224–237) · shuffle multi — `shuffleItems` (242–247) · redownload — `rdDocs`, `rdParts`, `rdSel` (250–258).

Cross-cutting handlers: `stageFiles` 305 · `updateFontSettings` 354 · `updateDownloadFormat` 363 · `finalizeDownload` 376 (DOCX → `downloadBlob` / PDF → `docxBlobToPdfBlob`) · `finalizeZipEntries` 387 · `changeMode` 397 · `backToHome` 408 · `handleAddMoreFiles` 427 · carry-over `carryToMode` 469 · `resetResults` 512.

Per-mode handlers: text flow — 520/532/549/568/580/599/1318/1557/1574/1590/1602/1611/1623 · shuffle docx — 629/712/985/1013/1042/1067/1099–1150 · shuffle multi — 1360/1452/1464/1470/1496/1527 · serial — 717/767/776/786/827/859/886/917/935/961 · redownload — 1155/1202/1211/1222/1231–1249/1265/1291.

Shuffle dispatch: `handleShuffle` (1347) = `shuffleItems` ? multi : docx ? docx : text.

Each playbook walks a real user action through the full code path. Read the line next to a step, open the file at that line, and you are standing exactly where a change would land.

4.1 · Shuffle mode — single .docx flow

1. **Upload** on UploadFirstCard → `stageFiles` (page 305) → `isValidDocxZip` (file-pipeline 33) rejects fake docx files with an English toast.
2. Click  **MCQ Shuffle** on ModeTabs → `changeMode` (397) → `carryToMode` (469) → `handleShuffleFiles` (1360) → single file → `handleDocxFile` (629) → `runFilePipeline` + `prepareShuffleXml` (125) strips color headers + non-MCQ lines → `parseDocxXml` (docx-xml 426) → state `docx`.
3. **DocxDetectCard**: selection buttons → `selectAll` / `selectNone` / `selectRange` (1099–1111); broken serial →  **Fix serial & download .docx (1..N)** → `handleDocxSerialFix` (1042) → `buildSerialFixedDocxBlob` (docx-exporter 190) → `repackDocxRemapped` → `finalizeDownload` (376); or enable **Run as-is**.
4. **ShuffleCard** →  **Shuffle & build sets** → `handleShuffle` (1347) → `handleDocxShuffle` (985) → `buildSets` (set-engine 46) → `setsDocx` → auto-scroll to results.
5. **DocxSetsResult**:  **Word (.docx) — renumbered / original** → `handleDocxDownload` (1013) → `buildShuffledDocxBlob` (170) → `buildShuffledXml` (90: XML surgery + renumber + refs) → `repackDocxRemapped` (font remap runs last) → `finalizeDownload`; if format is PDF → `docxBlobToPdfBlob` (pdf-export 80).
6. Color-header files additionally show **ColorShuffleInfoCard** (jump to serial mode via `openInSerialMode` 961) and **BlockedLinesCard**.

4.2 · Shuffle mode — multi-file flow

1. `handleShuffleFiles` (1360) with ≥ 2 files (cap 50 — `SHUFFLE_MAX_FILES`, limits.ts 30; over-cap toast at 1371) → `runFilePipeline` + `prepareShuffleXml` per file → `shuffleItems`.
2. Inline files Card (1864) + **MultiFileList**: reorder / remove → `reorderShuffleItems` (1452) / `removeShuffleItem` (1464, clears previous results).
3. **ShuffleCard** → `handleMultiShuffle` (1470): per-file `buildSets` → `shuffleMultiSets`.
4. **MultiDownloadCard — Download as one file (.docx)** → `handleMultiMergedDownload` (1496): per file `buildShuffledXml` → `replaceDocumentXml` (multi-docx 268) → `buildMergedDocxBlob` (414, one container-level font remap) → finalize. **Download separately (.zip)** → `handleMultiZipDownload` (1527): per-file `replaceDocumentXml(file, xml, fontSettings)` → `finalizeZipEntries` (387) → `buildZipBlob` (556) → `MCQ-shuffled-files.zip`.

4.3 · Shuffle mode — text / paste flow (also .txt)

1. UploadFirstCard paste tab → textarea → `handleTextChange` (520; autosave `mcq-shuffler-text`, debounced 600 ms) or .txt upload → `loadAndDetect` (549) → `runTextParse` (532) → `parseMcq` (parser 141) → auto-jumps into the shuffle work view.

2. **DetectCard**: selection /  **Fix numbering automatically** → `handleAutoFix` (580) → `autoFixNumbering` (parser 223).
3. **ShuffleCard** → `handleTextShuffle` (1318) → `buildSets` → `sets`.
4. **SetsResult**:  **Word (.docx) — one set per page** → `handleExportDocx` (1574) → `buildSetsDocxBlob` (202; `maybeRemapPackedDocx` 192 applies the font remap) → `finalize`; **.doc (legacy Word)** → `handleExportDoc` (1590) → `exportDocHtml` (299);  **Print** → `handlePrint` (1602) → `printSets` (306); copy buttons → `handleCopySet` (1611) / `handleCopyAll` (1623).

4.4 • Serial mode — file flow

1.  **MCQ Serial** → `SerialInputCard` dropzone → `loadSerialFiles` (717) → `runFilePipeline` + `analyzeColorDocx` (color-serial 231) → `serialDocs` (schemes default to continuous).
2. Exactly 1 file **with** colors → **ColorSerialCard**: pick a color chip or **Continuous across the file** → **Download serial .docx (B1)** → `handleColorSerial` (786) → `planSerialByColor` (472) → `applyColorSerialXml` (648) + `repackDocxRemapped` → `finalize`.
3. Exactly 1 file **without** colors → **NoColorSerialCard** → **Download continuous 1..N serial .docx** → same `handleColorSerial` with `{kind:"continuous"}`.
4. ≥2 files → **MultiSerialSchemeCard** (per-file scheme → `setSerialSchemes` 1821) + **MultiDownloadCard**: merged → `handleSerialMultiMerged` (827) → per-file plan/apply → `buildMergedDocxBlob(items, fontSettings)` → `finalize` (name "{base} (merged serial).docx"); ZIP → `handleSerialMultiZip` (859) → `MCQ-serial-files.zip`.

4.5 • Serial mode — paste flow

1. `SerialInputCard` Paste tab →  **Detect questions** → `handleSerialPasteDetect` (886) → `parseMcq`; clears the file list (mutually exclusive).
2. **SerialPasteCard**:  **Fix numbering automatically** → `handleSerialPasteFix` (917) → `autoFixNumbering`; **Download serial .docx (1..N)** → `handleSerialPasteDownload` (935) → `renumberQuestionsByPosition` (serial-paste 17) → `buildSetsDocxBlob(..., {includeHeader:false, includeSetHeader:false, fileName:"MCQ-Serial-Nq.docx"}, fontSettings)` → `finalize`.

4.6 • Redownload mode

1.  **MCQ Redownload** → `RedownloadInputCard` → `loadRedownloadFiles` (1155) → `runFilePipeline` with `parseRedownloadXml` (redownload 259) + `extractWatermark` (761) → `rdDocs`; all questions default-selected (1182).
2. **RedownloadPartsCard**: six checkboxes → `setRdParts` (1712); **Renumber serials 1, 2, 3...** → `setRdRenumber`.
3. **RedownloadQuestionsCard** per file: Select all / Deselect / Select range / checkboxes → page 1231–1246.
4. **MultiDownloadCard** "4. Download...": **Download as one file (.docx)** → `handleRdMerged` (1265) → `buildRdItems` (1249) → `buildRedownloadXml` (511) → single file: `replaceDocumentXml` / many: `buildMergedDocxBlob` → `finalize` (names "{base}

(redownload).docx" / "MCQ-Redownload-merged.docx"); **Download separately (.zip)** → `handleRdZip` (1291) → `MCQ-Redownload.zip`.

PART 5 Where to change common things — quick recipes

The lookup table below covers the edits you will most often ask for. The last column lists what must stay untouched so the edit stays safe — and, when a visible label changes, remember to sync the E2E selectors (Appendix) in the same commit.

I WANT TO CHANGE...	OPEN THIS FILE	TOUCH THIS	DO NOT TOUCH
Site font everywhere	<code>src/config/theme.ts</code>	<code>fonts.sans</code> (147); web-font recipe in the bottom comment (a)	<code>globals.css</code> font chain, <code>theme-css.ts</code>
Brand color (the green)	<code>src/config/theme.ts</code>	<code>colors.light.brand</code> 600/700 (+neighbors, 181–193); recipe (b)	<code>--st-*</code> var chain, component classes
Popup corners / toast shadow	<code>src/config/theme.ts</code>	<code>popup</code> object (254–261); recipe (c)	<code>ui/toast.tsx</code> / <code>ui/select.tsx</code> wiring
Corner radius of everything	<code>src/config/theme.ts</code>	<code>radius.base</code> (245) — one knob	sm/md/lg/xl (derived)
A card's title text	that card's component (Part 3 lists the exact line)	the <code><CardTitle></code> string	<code>page.tsx</code> props — unless the title is passed in (MultiDownloadCard titles come from page 1737/1824/1915)
A button label	that card's component	the <code><Button></code> text (many labels swap on .docx/.pdf — keep the <code>format === "pdf" ? ...</code> template)	E2E selectors in <code>scripts/e2e-*.ts</code> — sync them in the same commit
Upload size limit (50 MB)	<code>src/lib/mcq/limits.ts</code>	<code>MAX_FILE_BYTES</code> (24) + <code>FILE_TOO_BIG_MSG</code> in <code>file-pipeline.ts</code> (24)	dropzone emit logic
Max files in shuffle (50)	<code>src/lib/mcq/limits.ts</code>	<code>SHUFFLE_MAX_FILES</code> (30)	mode-work-bar cap logic
The 3 mode names / emoji / icons	<code>src/lib/mcq/mode-meta.ts</code>	<code>MODE_META</code> (25–53): title / tabTitle / description / emoji / icon	the id strings & <code>localStorage</code> <code>mcq-shuffler-mode</code> values
Font choices in "Fonts in the output file"	<code>src/lib/mcq/font-remap.ts</code>	<code>FONT_CHOICES</code> (63–67)	nothing — <code>sanitizeFontSettings</code> (page 107) reads this list automatically
Default remap fonts	<code>src/lib/mcq/font-remap.ts</code>	<code>DEFAULT_FONT_REMAP_SETTINGS</code> (55–60)	repack-docx wiring
PDF page cap (300) / quality	<code>src/lib/mcq/pdf-export.ts</code>	<code>PDF_MAX_PAGES</code> (40), scale rule (157), JPEG quality (199/215)	the DOCX path

Download format default (DOCX)	<code>src/app/page.tsx</code>	<code>DOWNLOAD_FORMAT_KEY</code> (120), initial state (212), hydrate effect (345)	DownloadFormatToggle component
Set-name style output (সেট A / ক / ১ / Set 1)	<code>src/lib/mcq/set-engine.ts</code>	<code>getSetName</code> (93) + option labels in sets-result.tsx (190–193)	exported-file samples are functional strings
Parser keywords / question-start regex	<code>src/lib/mcq/parser.ts (Q_RE)</code> · <code>docx-xml.ts</code>	⚠ functional Bengali patterns — change only together with the unit tests (test-mcq.ts, test-docx.ts)	coupling with <code>MAX_SERIAL_NUMBER</code> (limits.ts 18)
.txt/.csv paste size cap (5 MB)	<code>src/components/mcq/upload-first-card.tsx</code>	<code>TEXT_MAX_BYTES</code> (24) + message (59)	—
Hero title / badges / footer	<code>src/app/page.tsx</code>	header 1639–1659, footer 2073–2077	layout.tsx metadata
Output file names	<code>src/app/page.tsx (+ docx-exporter.ts)</code>	merged/zip name templates (page 843/871/1305/1544/1278; suffixes 182/205)	finalizeDownload / finalizeZipEntries
localStorage keys	<code>src/app/page.tsx</code>	<code>STORAGE_KEY</code> / <code>MODE_KEY</code> / <code>FONT_SETTINGS_KEY</code> / <code>DOWNLOAD_FORMAT_KEY</code> (97–120)	hydration effects
Format-kept-exact zip surgery	<code>repack-docx.ts</code> · <code>multi-docx.ts</code> · <code>docx-xml.ts</code>	mothers — edit only with the full test suite green	anything, without running scripts/test-*.ts

Before you push — the safety net is: `npx tsc --noEmit` · `npx eslint src` · unit tests `bun run scripts/test-mcq.ts` (+ test-docx, test-color-serial, test-multi-docx, test-redownload, test-reference, test-font-remap — 530+ assertions) · E2E `bun run scripts/e2e-*.ts` (10 suites) · static export `STATIC_EXPORT=1` `NEXT_PUBLIC_BASE_PATH=/MCQ_Shuffler_Pro` `npx next build` → `bash scripts/deploy-pages.sh <token>`.

Appendix — quick E2E anchor cross-reference

Selectors the automated tests depend on — if you rename one of these labels, update the matching selector in the same commit:

E2E SCRIPT	ANCHORS USED
<code>scripts/e2e-modes.ts</code>	<code>#step-result</code> , <code>#step-shuffle</code> , <code>#step-rd-questions</code> , "Download as one file (.docx)", "Download separately (.zip)", <code>button[aria-label="Move up"]</code> , <code>button[aria-label="Remove from list"]</code> , "Select all" (exact) / "Deselect all", <code>input[placeholder="From"/"To"]</code> , "Select range", "Word (.docx)", "1/50 files", <code>button[role="tab"]:has-text("MCQ Shuffle")</code> .
<code>scripts/e2e-pdf-export.ts</code>	<code>data-testid="download-format"</code> , <code>format-docx</code> , <code>format-pdf</code> .

scripts/e2e-font-remap.ts	data-testid="font-settings-card" , select ids font-unicode / font-bijoy , "Use fonts".
other suites	e2e-mode-tabs · e2e-multi-file · e2e-shuffle-headers · e2e-shuffle-limit · e2e-reference · e2e-live-bare-ref · e2e-back-home — mode tabs, staged texts, work-bar and card titles.

What shipped in this edition (v1.0 of the guide)

FEATURE	WHERE IT LIVES NOW
English UI (all cards, toasts, labels)	every component in src/components/mcq/ ; lang="en" in layout.tsx; E2E selectors synced in scripts/e2e-*.ts
Font remap in output files	engine src/lib/mcq/font-remap.ts + card font-settings-card.tsx wired into all 11 export call sites via repackDocxRemapped
DOCX / PDF download toggle	download-format-toggle.tsx + pdf-export.ts (docx-preview → html2canvas-pro → jsPDF, A4 slicing, 300-page cap)
One-place site theme	src/config/theme.ts → CSS vars → Tailwind brand utilities (246 classes migrated)
Multi-agent review fixes	destructive-foreground token, PDF page slicing, staging-time zip validation, toast feedback for rejected files, a11y + hydration guards

Change one card. Never the whole project.

That is what the mother / child architecture buys you: cards live in their own files, buttons trace to named handlers, shared behavior lives in the mothers of Part 2, and the whole site's look obeys one file — **src/config/theme.ts**.

Keep the test suite green, keep the E2E selectors in sync, and every future edit stays a five-minute job.